SENIOR ARCHITECT MASTERCLASS

Infrastructure

7 Topics across 7 Sections — Teach · Tips · Pocket Notes · Interview · Code · Output

Learn + Code | Pocket Notes | Interview Prep

Inference Engines

vLLM, TGI, TensorRT-LLM, Ollama, llama.cpp

> Imagine you have a brilliant chef (your LLM model). The chef is the same regardless of the kitchen they work in. But the KITCHEN matters: a professional kitchen with industrial ovens serves 100 meals/hour. A home kitchen serves 5. Inference engines are the kitchen -- they determine how fast and efficiently your model serves predictions. vLLM is the professional kitchen.

1. 5 Engines Compared

📖 DEEP TEACH

```
5 KITCHENS FOR YOUR LLM CHEF:

vLLM (THE PROFESSIONAL KITCHEN -- production standard):
Imagine a restaurant kitchen designed for maximum throughput. Orders come in constantly. The kitchen handles multiple orders at once, reuses prepped ingredients, and never lets a burner sit idle.
vLLM does this for LLMs: handles many requests simultaneously, reuses cached computations, and keeps the GPU busy 100% of the time.
Result: 2-4x more orders served per hour compared to a basic kitchen.
Use for: any production deployment. THE default choice.

TGI (Text Generation Inference -- the Docker-friendly kitchen):
Imagine a kitchen that comes in a shipping container. Plug it in and it works. Built by HuggingFace. Great integration with their model hub.
Use for: quickly deploying HuggingFace models when you want Docker simplicity.

TENSORRT-LLM (NVIDIA's Formula 1 kitchen):
Imagine a kitchen built by the company that makes the ovens (NVIDIA). They know every trick to squeeze maximum performance from their hardware. 30-50% faster than vLLM on the same ovens.
But: complex to set up. Like a Formula 1 car -- fastest but needs a pit crew.
Use for: when every millisecond and every request-per-second matters.

OLLAMA (the home kitchen):
Imagine running a model on your laptop with ONE command: ollama run llama3
Perfect for trying recipes at home (development, demos). NOT for a restaurant (production).
Use for: local development, testing, demos.

LLAMA.CPP (the camping stove -- works without electricity):
Imagine cooking without a gas line or electric stove. Just a portable burner.
llama.cpp runs LLMs on your CPU -- no GPU needed! A 7B model runs on a laptop. A 70B model runs on a Mac with 64GB RAM.
Use for: when you have no GPU at all. Edge devices. Offline scenarios.

DECISION:
Production with GPUs -> vLLM (90% of cases)
Quick Docker deploy -> TGI
Maximum speed on NVIDIA -> TensorRT-LLM
Local development -> Ollama
No GPU available -> llama.cpp
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of choosing an engine like choosing a vehicle for a delivery service. vLLM = delivery van (reliable, handles volume). TGI = rental van (quick to get). TensorRT = race car (fastest but needs a mechanic). Ollama = bicycle (personal use). llama.cpp = walking (works anywhere, no vehicle needed).

1. vLLM is your default. Best balance of speed and ease of use.
2. Do NOT use Ollama in production. It is designed for one person, not thousands.
3. TensorRT-LLM: 30-50% faster but 10x harder to set up. Only worth it if throughput is your bottleneck.
4. llama.cpp: surprisingly fast on Apple Silicon Macs (uses Metal GPU acceleration).
5. LiteLLM: not an engine but a TRANSLATOR. It gives you ONE interface that talks to ANY engine. Switch engines without changing your code.
6. ALL engines speak the same language (OpenAI-compatible API). Your client code works with any engine.
```

WAR STORY: Team used raw HuggingFace transformers to serve their model. 15 requests per second. Switched to vLLM: 55 requests per second. Same model, same GPU. The only change was the kitchen -- the chef stayed the same.

📋 POCKET NOTES

vLLM: professional kitchen. 2-4x throughput. PRODUCTION STANDARD.
TGI: Docker-friendly HuggingFace kitchen. Quick deploy.
TensorRT-LLM: NVIDIA Formula 1. 30-50% faster. Complex setup.
Ollama: home kitchen. One command. NOT for production.
llama.cpp: camping stove. CPU only. No GPU needed.
Decision: prod=vLLM | docker=TGI | max speed=TensorRT | local=Ollama | no GPU=llama.cpp
LiteLLM: translator proxy. One interface for any engine.
All engines speak OpenAI-compatible API.
pip install vllm | ollama.com/download

🎯 INTERVIEW PREP

SAY THIS: 'vLLM is my default for production. PagedAttention gives 2-4x throughput, continuous batching keeps GPUs busy, and the OpenAI-compatible API means zero client code changes. For local dev, Ollama.'

Q: vLLM vs TGI?
A: 'vLLM for maximum throughput and features like LoRA hot-swapping. TGI for quick Docker deployment of HuggingFace models. vLLM is generally faster.'

Q: When would you use llama.cpp?
A: 'No GPU available, edge devices, or offline scenarios. On Apple Silicon Macs it is surprisingly fast. Also for GGUF quantized models.'

⌨️ CODE

Python

```python
# vLLM: production serving (the professional kitchen)
from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Llama-3-8B-Instruct",
    gpu_memory_utilization=0.9,  # Use 90% of GPU
    max_model_len=4096,
)

params = SamplingParams(temperature=0.7, max_tokens=256)
outputs = llm.generate(["What is machine learning?"], params)
print(outputs[0].outputs[0].text)

# As OpenAI-compatible server (production):
# python -m vllm.entrypoints.openai.api_server --model meta-llama/Llama-3-8B-Instruct --port 8000

# Then use standard OpenAI client (your code does not change!):
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1", api_key="dummy")
resp = client.chat.completions.create(
    model="meta-llama/Llama-3-8B-Instruct",
    messages=[{"role":"user","content":"What is ML?"}]
)
print(resp.choices[0].message.content)

# LiteLLM: one interface for ANY engine/provider
import litellm
resp = litellm.completion(
    model="ollama/llama3",  # or gpt-4, claude-3, together_ai/...
    messages=[{"role":"user","content":"Hello"}]
)
print(f"LiteLLM: {resp.choices[0].message.content}")
```

📺 OUTPUT

```
ML is a subset of AI that enables systems to learn from data.

# vLLM: 55 req/s vs HuggingFace 15 req/s. Same model. Same GPU.
# Same OpenAI client code works with vLLM, TGI, or any engine.
# LiteLLM: one interface for every provider. Switch = change one string.
```

Optimization Techniques

KV-cache, batching, speculative decoding, PagedAttention

> Imagine a restaurant where the chef starts cooking every dish from scratch for every customer -- even if 10 people ordered the same pasta. Insane, right? You would prep the sauce in advance, cook multiple orders at once, and have a junior chef start plating while the senior chef finishes cooking. These 4 optimizations do exactly that for LLMs.

2. KV-Cache + Continuous Batching + Speculative Decoding + PagedAttention

📖 DEEP TEACH

```
4 TRICKS THAT MAKE PRODUCTION SERVING 2-4X FASTER:

1. KV-CACHE (remembering what you already read):
Imagine writing a long essay. Without KV-cache, every time you write a NEW word, you have to re-read the ENTIRE essay from the
beginning to remember the context. For a 1000-word essay, writing word 1000 means re-reading all 999 previous words. Insanely slow.

With KV-cache: you REMEMBER what you already read. Each new word only needs to look at what is new, not re-read everything. Like
having perfect memory instead of amnesia.

Result: generating long responses goes from painfully slow to fast.

2. CONTINUOUS BATCHING (no idle time):
Imagine a restaurant with 10 tables. Naive approach: wait until all 10 tables order, cook all 10 meals, serve all 10 at once. But
table 1 orders a salad (fast) and table 10 orders a steak (slow). The salad customer waits for the steak to finish. GPU sits idle
while the salad is ready.

Continuous batching: as SOON as the salad is done, serve it AND take a new order for that slot. The kitchen never has an empty
burner.

Result: 2-3x more meals served per hour. The GPU is ALWAYS working.

3. SPECULATIVE DECODING (the sous chef trick):
Imagine the head chef (big expensive model) is very careful and slow. A junior chef (small cheap model) is fast but sometimes
wrong.

Speculative decoding: the junior chef quickly prepares 5 components. The head chef checks all 5 at once (much faster than doing
them one by one). If the junior was right (usually 70-80% of the time), the head chef approves all 5 instantly. If wrong, the head
chef corrects only the wrong parts.

Result: 2-3x faster because the head chef verifies in bulk instead of doing everything from scratch.

4. PAGEDATTENTION (vLLM's secret weapon):
Imagine a parking lot. Traditional approach: each car reserves a WHOLE ROW of spaces, even if they only need 3 spaces. Massive
waste -- most rows are half-empty.

PagedAttention: allocate spaces on demand, one at a time. No wasted spaces. Same parking lot fits 2-4x more cars.

This is specifically about how GPU memory is used for the KV-cache. Traditional = wasteful contiguous blocks. Paged = efficient on-
demand pages. This is WHY vLLM beats other engines.

All 4 are built into vLLM. Use vLLM = get all 4 optimizations for free.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of these like restaurant efficiency upgrades:
  KV-cache = prep station (do not re-chop vegetables for every dish)
  Continuous batching = always have something on every burner
  Speculative decoding = junior chef preps, senior chef approves in bulk
  PagedAttention = flexible seating instead of fixed reserved tables

1. You do NOT configure these manually. vLLM enables all 4 automatically.
2. Speculative decoding: needs a good draft model. Llama-3-1B as draft for Llama-3-8B.
3. Max_model_len: set this to the MINIMUM you need. Shorter context = more concurrent users.
```

4. GPU memory utilization: vLLM default is 0.9 (90%). Push to 0.95 if the GPU is dedicated.

WAR STORY: Team served a 70B model. Maximum 8 users at the same time. Switched to vLLM (PagedAttention): 32 concurrent users on the same GPU. 4x improvement from smarter memory management alone. No hardware upgrade needed.

📋 POCKET NOTES

KV-Cache: remember what you already read. No re-reading entire essay.
Continuous Batching: as one request finishes, slot in new one. GPU never idle. 2-3x.
Speculative Decoding: fast junior model proposes, slow senior verifies in bulk. 2-3x.
PagedAttention: flexible memory allocation. 2-4x more concurrent users. vLLM exclusive.
All 4 built into vLLM. Use vLLM = get all free.
Max_model_len: shorter = more concurrent requests.
GPU utilization: 0.9 default, 0.95 if dedicated.
Draft model for speculative: small model that matches the big one.

🎯 INTERVIEW PREP

SAY THIS: 'Production inference combines four optimizations: KV-cache avoids redundant computation, continuous batching keeps GPUs fully utilized, speculative decoding gets 2-3x generation speedup, and PagedAttention enables 2-4x more concurrent requests. vLLM implements all four.'

Q: How does speculative decoding maintain quality?
A: 'The large model VERIFIES every token. Wrong tokens from the draft model get corrected. Output is mathematically identical to running the large model alone. Speed without quality loss.'

Q: Why is PagedAttention a big deal?
A: 'KV-cache for a 70B model can use 40GB per request. Traditional allocation wastes memory through fragmentation, limiting to 8 concurrent requests. PagedAttention eliminates fragmentation, fitting 32+ in the same memory.'

⌨️ CODE

Python

```python
# vLLM with all 4 optimizations (all automatic)
from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Llama-3-8B-Instruct",
    gpu_memory_utilization=0.9,  # Use 90% of GPU RAM
    max_model_len=4096,          # Shorter = more concurrent users
    # All optimizations enabled automatically:
    # KV-cache: on
    # Continuous batching: on
    # PagedAttention: on (this is vLLM's core innovation)
    # Speculative decoding (optional):
    # speculative_model="meta-llama/Llama-3-1B",
    # num_speculative_tokens=5,
)

prompts = [
    "Explain quantum computing in one sentence.",
    "What is the capital of France?",
    "Write a haiku about coding.",
]
params = SamplingParams(temperature=0.7, max_tokens=100)
outputs = llm.generate(prompts, params)

for i, output in enumerate(outputs):
    print(f"Prompt {i+1}: {output.outputs[0].text[:60]}...")

print(f"\
3 prompts processed SIMULTANEOUSLY via continuous batching.")
print(f"PagedAttention: 2-4x more requests fit in same GPU memory.")
```

📺 OUTPUT

```
Prompt 1: Quantum computers use qubits that can exist in multiple states...
Prompt 2: The capital of France is Paris...
Prompt 3: Code flows like streams / Bugs hide in nested loops deep / Stack...

3 prompts processed SIMULTANEOUSLY via continuous batching.
PagedAttention: 2-4x more requests fit in same GPU memory.
# All optimizations working automatically. Zero configuration needed.
```

Quantization

GPTQ, AWQ, GGUF, bitsandbytes — shrink models

> Imagine a beautiful 4K movie that is 50GB. You want to watch it on your phone but only have 10GB of space. You compress it to a smaller file -- it looks 97% as good, and now it fits. Quantization does the same for LLM models: compresses them to use less GPU memory with minimal quality loss. A 70B model shrinks from 140GB to 35GB.

3. 4 Methods + Decision Guide

📖 DEEP TEACH

```
THE PROBLEM:
LLM models store each of their billions of numbers as a 16-bit value (2 bytes). A 70 billion parameter model: 70B x 2 bytes = 140
GB. That needs 2 expensive A100 GPUs (80GB each).

THE FIX -- QUANTIZATION:
Compress each number from 16 bits to 4 bits (half a byte). Same 70B model: 70B x 0.5 bytes = 35 GB. Fits on ONE A100! Quality loss:
about 1-3%. Like the 4K-to-1080p compression -- you barely notice the difference.

4 COMPRESSION METHODS:

GPTQ (the GPU standard):
Imagine compressing a movie with a smart algorithm that preserves the important scenes in high quality and compresses the boring
parts more aggressively. GPTQ analyzes which weights matter most and preserves those.
Use for: serving on GPUs. Widely supported by vLLM, TGI.

AWQ (the quality-focused option):
Imagine a smarter version of GPTQ. It identifies which weights are ACTIVATED most often and preserves those with higher precision.
Slightly better quality than GPTQ at the same compression level.
Use for: when quality matters more than compatibility.

GGUF (the CPU format):
Imagine a movie format optimized for playing on your phone instead of a TV. GGUF is optimized for CPU and Apple Silicon. Used by
llama.cpp and Ollama.
Comes in quality levels: Q2 (most compressed), Q4 (balanced), Q5, Q8 (least compressed).
Q4_K_M is the sweet spot for most people.
Use for: local inference, no GPU, Ollama.

BITSANDBYTES (the training format):
Specifically designed for QLoRA training (not production serving).
Compresses the model so you can TRAIN on it with limited memory.
Use for: fine-tuning only. For serving, use GPTQ or AWQ.

QUALITY VS SIZE:
  16-bit: 100% quality, 140 GB for 70B model
  8-bit:  99% quality, 70 GB
  4-bit:  97% quality, 35 GB
  3-bit:  93% quality, 26 GB (usually too much loss)

GOLDEN RULE:
A quantized BIG model beats a full-precision SMALL model.
70B at 4-bit (97% quality of a genius) > 13B at full precision (100% quality of a mediocre thinker).
Always quantize the BIGGEST model that fits your hardware.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like choosing between a compressed 4K movie on a big TV vs an uncompressed standard-definition movie on a small TV. The
compressed 4K looks way better despite the compression.

1. ALWAYS benchmark quantized vs full on YOUR data. The 1-3% quality loss varies by task.
2. AWQ > GPTQ in quality at the same compression. Use AWQ when available.
3. GGUF Q4_K_M: best balance for local/CPU inference.
4. Do NOT compress below 4-bit for production. Quality drops too steeply.
5. Pre-quantized models on HuggingFace: search for TheBloke or unsloth repos. Do NOT quantize yourself unless you have a specific
```

reason.
6. The BIGGEST mistake: running a full-precision small model instead of a quantized large model.

WAR STORY: Team ran full-precision 13B model on an expensive A100. Competitor ran quantized 70B (AWQ 4-bit) on the same GPU. The competitor's model was dramatically better because a 97%-quality genius beats a 100%-quality average thinker.

📋 POCKET NOTES

GPTQ: GPU production. Smart compression. Widely supported.
AWQ: GPU production. Better quality than GPTQ at same size.
GGUF: CPU/Ollama format. Q4_K_M best balance.
bitsandbytes: for QLoRA TRAINING, not production serving.
Quality: 16bit=100% | 8bit=99% | 4bit=97% | 3bit=93%
Size: 70B fp16=140GB | 8bit=70GB | 4bit=35GB
Golden rule: quantized 70B (97% genius) > full 13B (100% average)
Decision: GPU prod=AWQ/GPTQ | CPU=GGUF Q4_K_M | training=bitsandbytes
Never go below 4-bit in production.
Pre-quantized on HuggingFace: TheBloke, unsloth repos.

🎯 INTERVIEW PREP

SAY THIS: 'I always quantize the largest model that fits. A 4-bit 70B significantly outperforms a full-precision 13B. For GPU production, AWQ for best quality. For local CPU, GGUF Q4_K_M. I always benchmark quantized vs full on my eval set.'

Q: How much quality do you lose?
A: 'Typically 1-3% on benchmarks. But it depends on the task -- some are more sensitive. I always test on MY eval set. If the drop exceeds 3%, I try 8-bit or find more GPU memory.'

Q: When would you NOT quantize?
A: 'When I have enough GPU memory for full precision and cost is not a concern. Or for tasks extremely sensitive to numerical precision. But in practice, quantization is almost always worth it.'

⌨️ CODE

Python

```python
# Quantization options

# Option 1: Load pre-quantized GPTQ model in vLLM
from vllm import LLM
llm = LLM(
    model="TheBloke/Llama-2-70B-Chat-GPTQ",
    quantization="gptq",
    tensor_parallel_size=1,  # Fits on 1 GPU now!
)
print("70B GPTQ: fits on 1 GPU (was 2 at full precision)")

# Option 2: bitsandbytes 4-bit for QLoRA training
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype="bfloat16",
    bnb_4bit_use_double_quant=True,
)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3-70B",
    quantization_config=bnb_config, device_map="auto")
print("BnB 4-bit: 70B loaded for QLoRA training")

# Option 3: GGUF via Ollama (one command!)
# ollama run llama3:70b-q4_K_M
print("\
GGUF: ollama run llama3:70b-q4_K_M (runs on CPU!)")

print("\
70B MODEL MEMORY:")
print("  Full (16-bit):  140 GB (2 expensive GPUs)")
print("  8-bit:           70 GB (1 expensive GPU)")
```

```
print("  4-bit:           35 GB (1 GPU or 2 consumer GPUs)")
print("  Quality loss:    ~1-3% at 4-bit. Barely noticeable.")
```

📺 **OUTPUT**

```
70B GPTQ: fits on 1 GPU (was 2 at full precision)
BnB 4-bit: 70B loaded for QLoRA training
GGUF: ollama run llama3:70b-q4_K_M (runs on CPU!)

70B MODEL MEMORY:
  Full (16-bit):  140 GB (2 expensive GPUs)
  8-bit:           70 GB (1 expensive GPU)
  4-bit:           35 GB (1 GPU or 2 consumer GPUs)
  Quality loss:    ~1-3% at 4-bit. Barely noticeable.
```

API Gateway Patterns

Fallbacks, load balancing, retries

> Imagine you order all your restaurant supplies from one distributor. They have a truck breakdown. Your restaurant shuts down. Stupid, right? You should have a backup distributor. API gateway patterns = having multiple LLM providers, automatic failover, and smart retry logic so your app NEVER goes down because one provider has an issue.

4. Multi-Provider + Fallbacks + Rate Limiting + Retries

📖 DEEP TEACH

```
4 PATTERNS (think of it as supply chain management for AI):

1. MULTI-PROVIDER (do not put all eggs in one basket):
Imagine having 3 food suppliers. If Supplier A runs out of tomatoes, you automatically order from Supplier B.

Route across OpenAI + Anthropic + your own local model. If OpenAI goes down, traffic automatically shifts. Your users never notice.

2. FALLBACK CHAINS (Plan A, Plan B, Plan C):
Primary: GPT-4o (best quality)
Fallback 1: Claude 3.5 (if OpenAI fails)
Fallback 2: local Llama-3-70B (if ALL cloud APIs fail)

Automatic failover. Like having a generator when the power goes out.

3. RATE LIMITING (do not eat the whole buffet):
OpenAI limits you to X requests per minute. Hit the limit = 429 error = your app crashes.

Solution: queue requests, spread across providers, and limit per user (20 requests/minute) to prevent one user from eating your
entire quota.

4. RETRY WITH EXPONENTIAL BACKOFF (try again, but wait longer each time):
Request fails? Wait 1 second, try again.
Fails again? Wait 2 seconds, try again.
Fails again? Wait 4 seconds, try again.
Still fails? Switch to fallback provider.

Add a little randomness (jitter) to each wait. Otherwise, 1000 users all retry at the exact same second and crash the server again
(thundering herd).

LITELLM:
A Python proxy that translates between 100+ LLM providers. ONE interface, ANY provider.
Built-in: fallbacks, load balancing, budget tracking, caching.
Like having a universal power adapter that works in every country.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like building a reliable postal service. You do not use ONE delivery company. You have a primary (FedEx), backup (UPS),
and emergency (your own van). Packages always arrive.

1. ALWAYS configure at least 2 providers. Primary + fallback.
2. LiteLLM is the standard proxy for multi-provider routing.
3. Exponential backoff: 1s -> 2s -> 4s with random jitter. Max 3 retries.
4. CACHE identical requests in Redis. If 100 people ask the same question, call the LLM once.
5. HEALTH CHECKS: ping each provider every 30 seconds. Remove unhealthy ones from rotation.
6. LOG which provider served each request. Essential for cost analysis and incident review.

WAR STORY: Production app used ONLY OpenAI. OpenAI had a 4-hour outage. App was completely down. Lost $200K in revenue. Added
Anthropic as fallback + local Llama as emergency. Next OpenAI outage: zero downtime. Fallback kicked in within 2 seconds.
```

📋 POCKET NOTES

Multi-provider: do not depend on one. Route across OpenAI + Anthropic + local.
Fallback chain: Plan A -> Plan B -> Plan C -> error.
Rate limiting: per-user (20/min) + global (stay under provider limits).
Retry: exponential backoff (1s, 2s, 4s) + random jitter + max 3 tries.
LiteLLM: universal proxy for 100+ providers. Production standard.
  Fallbacks, load balancing, budget tracking, caching.
Health checks: ping providers every 30s.
Cache identical requests (Redis) to save cost.
ALWAYS have 2+ providers configured.

🎯 INTERVIEW PREP

SAY THIS: 'I design with multi-provider resilience. LiteLLM as gateway proxy, automatic failover chains, exponential backoff with jitter, and per-user rate limiting. Single provider outage = zero user impact.'

Q: OpenAI goes down. What happens?
A: 'Automatic failover to Anthropic within 2 seconds. If both down, route to self-hosted Llama. Users experience slightly different quality but zero downtime.'

Q: How to handle rate limits?
A: 'Three layers: per-user limit prevents abuse, global rate limiter stays under provider limits, and if approaching limits, queue requests rather than dropping them.'

⌨️ CODE

Python

```python
import litellm
from litellm import completion
import time, random

# Fallback chain: try providers in order
def resilient_call(messages):
    models = ["gpt-4o", "claude-3-5-sonnet-20241022", "ollama/llama3"]
    for i, model in enumerate(models):
        try:
            start = time.time()
            response = completion(model=model, messages=messages, timeout=10)
            ms = (time.time()-start)*1000
            print(f"  Provider {i+1} ({model}): OK in {ms:.0f}ms")
            return response
        except Exception as e:
            print(f"  Provider {i+1} ({model}): FAILED")
            if i < len(models)-1: print(f"  Falling back...")
    raise Exception("All providers failed")

# Exponential backoff with jitter
def retry_with_backoff(func, max_retries=3):
    for attempt in range(max_retries):
        try: return func()
        except Exception as e:
            if attempt == max_retries-1: raise
            delay = (2**attempt) + random.uniform(0, 0.5)
            print(f"  Retry {attempt+1} in {delay:.1f}s...")
            time.sleep(delay)

# Rate limiter (token bucket)
class RateLimiter:
    def __init__(self, rpm=20):
        self.rpm = rpm; self.calls = []
    def allow(self):
        now = time.time()
        self.calls = [t for t in self.calls if now-t < 60]
        if len(self.calls) >= self.rpm: return False
        self.calls.append(now); return True

print("RESILIENT CALL:")
msg = [{"role":"user","content":"What is 2+2?"}]
result = resilient_call(msg)
```

```
print(f"  Answer: {result.choices[0].message.content}")

limiter = RateLimiter(rpm=20)
print(f"\
Rate limiter: {'Allowed' if limiter.allow() else 'Blocked'}")
```

📺 **OUTPUT**

```
RESILIENT CALL:
  Provider 1 (gpt-4o): OK in 850ms
  Answer: 4

Rate limiter: Allowed

# If GPT-4o fails: auto-fallback to Claude, then local Llama.
# Backoff: 1s, 2s, 4s delays with jitter. No thundering herd.
# Rate limiter: 20 requests per minute per user.
```

Model Routing

Cheap model for simple, expensive for complex

> Imagine a hospital emergency room. A patient with a paper cut does not need the head surgeon -- a nurse can handle it. A patient with a heart attack needs the best doctor available. Model routing = sending simple questions to cheap/fast models and complex questions to expensive/smart models. This cuts costs 50-70% with zero quality loss on simple queries.

5. Complexity-Based Routing + Cascading

📖 DEEP TEACH

```
THE IDEA:
Not every question needs the smartest (most expensive) model.

'What time is it?' -> GPT-3.5 at $0.001/call (nurse handles it)
'Design a microservices architecture for...' -> GPT-4 at $0.030/call (surgeon needed)

If 70% of your questions are simple, routing saves 70% x ($0.030 - $0.001) = massive savings.

2 APPROACHES:

1. COMPLEXITY ROUTER (classify first, then route):
Imagine a hospital receptionist who asks 'what is your problem?' and sends you to the right department.

A small classifier checks each question: simple or complex?
  Simple -> cheap model (GPT-3.5, fast and cheap)
  Complex -> expensive model (GPT-4, smart and thorough)

The classifier can be: keyword rules ('compare', 'analyze', 'design' = complex), query length (longer = usually more complex), or a small ML model.

2. CASCADING (try cheap first, escalate if unsure):
Imagine a customer service center. Junior agent answers first. If they are unsure, they transfer to a senior agent.

Step 1: Send to GPT-3.5.
Step 2: Check confidence. Does the answer contain hedging words ('maybe', 'I think', 'not sure')?
Step 3: If low confidence -> re-send the SAME question to GPT-4.

Most questions get answered by the junior agent. Only the tricky ones escalate. The user sees ONE answer -- they never know about the routing.

COST MATH:
1000 queries/day, ALL to GPT-4: $30/day
1000 queries/day, 70% to GPT-3.5 + 30% to GPT-4: $9.70/day
Savings: 68%. Quality unchanged for the 70% simple queries.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like a law firm. The paralegal handles routine paperwork (cheap). The partner handles complex cases (expensive). You do not pay partner rates for filing paperwork.

1. START SIMPLE: if query is shorter than 50 words, use cheap model. Iterate from there.
2. Cascading is SAFER than routing because the cheap model tries first -- if it works, great. If not, the expensive model catches it.
3. DOMAIN OVERRIDE: medical, legal, financial questions ALWAYS use the best model. Safety first. You do not route heart attacks to the nurse.
4. A/B TEST your routing: randomly route 50% of traffic without routing, compare quality. Make sure routing does not hurt.

WAR STORY: SaaS app routed ALL queries to GPT-4. $15K/month. Analysis showed 75% were simple lookups ('what is my plan?', 'how to reset password?'). Routed those to GPT-3.5. Cost dropped to $4K/month. Quality scores unchanged for simple queries.
```

📋 POCKET NOTES

Complexity Router: classify question -> route to right model
Cascading: try cheap first -> escalate if uncertain
Analogy: hospital ER (nurse for paper cuts, surgeon for heart attacks)
Cost math: 70% routed to cheap = 68% savings
Classify by: keywords, query length, small classifier model
Confidence check: hedging words = escalate ('maybe', 'not sure')
Domain override: medical/legal = ALWAYS best model
Start: simple rules (query length). Iterate with data.
A/B test routing to verify quality maintained.

🎯 INTERVIEW PREP

SAY THIS: 'I implement cost-optimized model routing. Simple queries go to GPT-3.5, complex ones to GPT-4. For safety-critical domains, always the best model. This typically saves 50-70% with zero quality loss on simple queries.'

Q: How do you build the classifier?
A: 'Start with rules: query length, presence of reasoning keywords. Graduate to a fine-tuned classifier when you have production data. Train on 1000 labeled examples from production logs.'

Q: What if cheap model gives a bad answer?
A: 'Cascading: cheap answers first, I check for confidence signals. Low confidence triggers escalation. User sees one answer -- they never see the routing.'

⌨️ CODE

Python

```python
from openai import OpenAI
client = OpenAI()

def classify_complexity(query):
    """Simple classifier (upgrade to ML model in production)"""
    complex_signals = ["compare","analyze","design","explain how",
                       "tradeoffs","step by step","architecture"]
    if any(s in query.lower() for s in complex_signals): return "complex"
    if len(query.split()) > 30: return "complex"
    return "simple"

def routed_call(query):
    complexity = classify_complexity(query)
    model = "gpt-3.5-turbo" if complexity=="simple" else "gpt-4"
    cost_rate = 0.001 if complexity=="simple" else 0.03

    r = client.chat.completions.create(
        model=model,
        messages=[{"role":"user","content":query}])
    tokens = r.usage.total_tokens
    cost = tokens/1000 * cost_rate
    print(f"  [{complexity}] -> {model} | {tokens} tok | ${cost:.4f}")
    return r.choices[0].message.content

print("MODEL ROUTING:")
for q in ["What is 2+2?",
          "What is the capital of France?",
          "Compare PostgreSQL and MongoDB for a social media app",
          "Design a microservices architecture for e-commerce"]:
    answer = routed_call(q)
    print(f"  Answer: {answer[:50]}...\
")
```

📺 OUTPUT

```
MODEL ROUTING:
  [simple] -> gpt-3.5-turbo | 45 tok | $0.0000
  Answer: 4...
```

```
  [simple] -> gpt-3.5-turbo | 38 tok | $0.0000
  Answer: Paris...

  [complex] -> gpt-4 | 320 tok | $0.0096
  Answer: PostgreSQL is better for structured data with ACID...

  [complex] -> gpt-4 | 450 tok | $0.0135
  Answer: A microservices architecture for e-commerce should...

# Simple questions: fast + cheap. Complex: smart + thorough.
```

Streaming + Containerization

SSE, Docker, Kubernetes

> Without streaming, the user stares at a blank screen for 5 seconds, then the full answer appears at once. WITH streaming, the first word appears in 300ms and the rest flows in word by word. Users perceive streaming as 3-5x faster even though the total time is the same. Containerization (Docker/K8s) makes your deployment reproducible and scalable.

6. SSE Streaming + Docker + K8s for LLM

📖 DEEP TEACH

STREAMING -- WHY USERS LOVE IT:
Imagine texting someone. You see the typing indicator and words appearing. Compare that to: silence for 30 seconds, then a wall of text. The first feels responsive. The second feels broken. Same total time, but perception is totally different.

SSE (Server-Sent Events) -- THE STANDARD:
The server pushes words to the client one at a time over HTTP. Like a news ticker scrolling across a TV screen.
All LLM providers (OpenAI, Anthropic) use SSE for streaming. Set stream=True and you get tokens one by one.

FIRST TOKEN LATENCY: The metric users actually FEEL. Target: under 500ms. This is how quickly the first word appears after the user hits send.

CONTAINERIZATION -- WHY YOU NEED IT:
Imagine trying to recreate your kitchen setup in a new building. 'I used Python 3.11 with CUDA 12.1 and that specific version of PyTorch...' One wrong version and nothing works.

Docker solves this: package your entire kitchen (code + dependencies + GPU drivers) into a shipping container. Ship it anywhere. It works the same everywhere.

DOCKER FOR LLMS:
- Use nvidia/cuda base image for GPU support
- Pass GPUs with: docker run --gpus all
- Health check endpoint: /health (loads model and runs one inference)
- Do NOT bake model weights into the image (too big). Download at startup or mount from shared storage.

KUBERNETES FOR SCALING:
Imagine you need 10 copies of your restaurant kitchen to handle dinner rush, but only 2 during lunch.
K8s automatically scales: more copies when busy, fewer when quiet.
- Autoscale on request QUEUE LENGTH (not CPU -- CPU is misleading for GPU workloads)
- GPU node pools: machines with A100/H100 GPUs
- Rolling updates: deploy new model version without downtime

PRODUCTION STACK:
vLLM (engine) -> Docker (package) -> Kubernetes (scale) -> Load Balancer -> LiteLLM (gateway) -> Client

🏗 ARCHITECT TIPS + WAR STORIES

Think of streaming like a waiter who brings courses as they are ready vs one who holds everything until the last course is done. Same total time. Completely different customer experience.

1. ALWAYS enable streaming in production. Users perceive it as 3-5x faster.
2. First token latency < 500ms is the target. This is what users feel.
3. Docker image tip: do NOT put model weights in the image (would be 35GB+). Mount them from shared storage or download at startup.
4. K8s autoscale on QUEUE LENGTH, not CPU. For GPU workloads, CPU is a misleading metric.
5. Health check: do NOT just check if the port is open. Actually run a small inference. If the model is not loaded, the health check should fail.

WAR STORY: Team deployed with 5-second average response time. Users complained 'it is slow.' Added streaming: same total time but first word appears in 300ms. User complaints dropped 90%. The food was not faster -- but the appetizer arrived immediately.

📋 POCKET NOTES

```
Streaming:
  SSE: server pushes words one at a time. THE standard.
  First token latency < 500ms (what users FEEL).
  Users perceive streaming as 3-5x faster (same total time).
  stream=True in all provider APIs.
Docker:
  nvidia/cuda base image for GPU
  --gpus all flag for GPU access
  Do NOT put model weights in image (too big). Mount them.
  Health check: /health that runs actual inference.
Kubernetes:
  Autoscale on queue length, NOT CPU.
  GPU node pools (A100, H100).
  Rolling updates: zero-downtime model deploys.
Stack: vLLM -> Docker -> K8s -> Load Balancer -> LiteLLM -> Client
```

🎯 INTERVIEW PREP

SAY THIS: 'I always deploy with streaming. First token latency under 500ms is the UX target. The stack is vLLM in Docker on Kubernetes, autoscaling on queue length, with LiteLLM as multi-provider gateway.'

Q: How do you scale LLM on Kubernetes?
A: 'GPU node pools with A100s. Autoscale on request queue length -- not CPU, which is misleading for GPU workloads. Rolling updates for model deploys. Health checks that run actual inference.'

Q: Docker image is 15GB. How to reduce?
A: 'Do not include model weights in the image. Download at startup or mount from shared storage. Multi-stage builds for code. Slim CUDA base images.'

⌨ CODE

Python

```python
# FastAPI + SSE Streaming
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from openai import OpenAI
import json

app = FastAPI()
client = OpenAI()

@app.get("/health")
async def health():
    return {"status": "healthy"}

@app.post("/chat")
async def chat(query: str):
    async def stream_tokens():
        response = client.chat.completions.create(
            model="gpt-4",
            messages=[{"role":"user","content":query}],
            stream=True  # Words flow one at a time
        )
        for chunk in response:
            if chunk.choices[0].delta.content:
                token = chunk.choices[0].delta.content
                yield f"data: {json.dumps({'token': token})}\
\
"
        yield "data: [DONE]\
\
"

    return StreamingResponse(stream_tokens(), media_type="text/event-stream")

print("""# Dockerfile for LLM service:
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04
RUN pip install vllm fastapi uvicorn
```

```
HEALTHCHECK CMD curl -f http://localhost:8000/health
CMD ["python", "-m", "vllm.entrypoints.openai.api_server",
     "--model", "meta-llama/Llama-3-8B-Instruct", "--port", "8000"]

# Run: docker run --gpus all -p 8000:8000 llm-service
# K8s: autoscale on queue length with GPU node pools""")
```

📺 **OUTPUT**

```
# Dockerfile for LLM service:
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04
RUN pip install vllm fastapi uvicorn
HEALTHCHECK CMD curl -f http://localhost:8000/health
CMD ["python", "-m", "vllm.entrypoints.openai.api_server"...]

# Run: docker run --gpus all -p 8000:8000 llm-service
# K8s: autoscale on queue length with GPU node pools

# Streaming: first word in <500ms. Users love it.
# Without streaming: 5 seconds of nothing. Users hate it.
```

Serverless / Managed Platforms

Bedrock, Azure OpenAI, Groq, Together AI

> Imagine the choice between building your own kitchen (self-hosted) vs ordering from restaurants (managed platforms). Building your own kitchen gives maximum control but needs equipment, maintenance, and a chef. Ordering from restaurants is easier but you are at their mercy for menu, pricing, and quality. Know when each makes sense.

7. 8 Platforms Compared

📖 DEEP TEACH

```
8 RESTAURANTS (managed platforms) vs COOKING AT HOME (self-hosted):

AWS BEDROCK (Amazon's restaurant):
Multiple chefs: Claude, Llama, Mistral, Titan. Fits into your existing AWS infrastructure.
Best for: teams already on AWS.

AZURE OPENAI (Microsoft's restaurant):
OpenAI's models with Microsoft's security and compliance. Your data stays in your Azure region.
Best for: enterprise on Azure who need OpenAI with compliance.

GCP VERTEX AI (Google's restaurant):
Gemini models + open-source options. Google Cloud integration.
Best for: teams already on Google Cloud.

GROQ (the speed restaurant):
Custom hardware (LPU) that serves 10-50x FASTER than GPU-based options. The food arrives almost instantly.
Limited menu (fewer models). But incredibly fast.
Best for: real-time apps where response time under 100ms matters.

TOGETHER AI (the budget restaurant with great food):
Open-source models at low prices. You can fine-tune AND the model is yours to keep.
Best for: cost-sensitive teams wanting open models.

FIREWORKS AI (similar to Together):
Fast inference, good function calling support.

REPLICATE (the food truck):
Run any model in a container. Pay per second.
Best for: experimentation, running unusual models.

MODAL (the catering service):
Serverless GPU functions. Pay per second. Good for batch jobs.
Best for: intermittent workloads that do not run 24/7.

WHEN MANAGED vs SELF-HOSTED:
Managed is cheaper when: under 1 million words/day (no idle GPU costs).
Self-hosted is cheaper when: over 1 million words/day (GPU runs constantly anyway).
Managed is required when: using proprietary models (GPT-4, Claude API).
Self-hosted is required when: data cannot leave your servers (healthcare, finance, government).

LOCK-IN PREVENTION:
Use LiteLLM as your gateway proxy. Your code calls LiteLLM. LiteLLM calls the provider. Switch providers = change one config line, zero code changes.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like choosing between meal delivery, restaurants, and cooking at home. Delivery (managed) is convenient for occasional meals. Restaurants (specialized platforms) for special occasions. Cooking at home (self-hosted) saves money when you cook every day.

1. Use YOUR CLOUD's offering: AWS=Bedrock, Azure=Azure OpenAI, GCP=Vertex. Simplest integration.
2. Groq: incredible speed but limited models. Perfect for real-time chat.
3. Together AI: best value for open-source models you can own.
```

4. LiteLLM proxy: prevents vendor lock-in. One interface, any provider.
5. Cost crossover: managed cheaper at < 1M tokens/day. Self-hosted cheaper above that.
6. PROVISIONED THROUGHPUT (Azure, Bedrock): predictable pricing for stable workloads. Like a fixed monthly subscription vs pay-per-use.

WAR STORY: Startup used Groq for ultra-fast chat (200ms responses). Users loved the speed. But Groq did not have their largest model. Solution: Groq for interactive chat, Together AI for background processing. LiteLLM routed automatically based on task type. Best of both worlds.

📋 POCKET NOTES

AWS Bedrock: multi-model, AWS-native. Enterprise compliance.
Azure OpenAI: OpenAI + Azure security. Data stays in your region.
GCP Vertex: Gemini + open models. GCP-native.
Groq: FASTEST (10-50x). Custom hardware. Limited models.
Together AI: open-source, cheap, you own the model.
Fireworks: fast open-source, function calling.
Replicate: any model in container. Pay per second.
Modal: serverless GPU. Batch/intermittent workloads.
Decision: use your cloud (AWS=Bedrock, Azure=AzureOAI, GCP=Vertex)
  Speed=Groq | Budget=Together | Experimental=Replicate
Crossover: managed cheaper <1M tok/day, self-host cheaper above.
LiteLLM: prevents lock-in. One interface, any provider.

🎯 INTERVIEW PREP

SAY THIS: 'I match platform to cloud infrastructure: Bedrock for AWS, Azure OpenAI for Azure. For speed-critical: Groq. For cost-sensitive open-source: Together AI. LiteLLM as abstraction layer to prevent lock-in.'

Q: Managed vs self-hosted decision?
A: 'Cost crossover at ~1M tokens/day. Below that, managed is cheaper. Above, self-hosted. Also: compliance requirements may mandate managed, and data privacy may mandate self-hosted.'

Q: How to prevent lock-in?
A: 'LiteLLM proxy. My code calls LiteLLM. LiteLLM calls the provider. Switching from OpenAI to Anthropic = one config change, zero code changes.'

⌨ CODE

Python

```python
# Multi-platform with LiteLLM (one interface for everything)
import litellm

def call_any(model, prompt):
    r = litellm.completion(
        model=model,
        messages=[{"role":"user","content":prompt}])
    return r.choices[0].message.content

# Same interface, ANY provider:
# call_any("gpt-4o", "Hello")                      # OpenAI
# call_any("claude-3-5-sonnet-20241022", "Hello")  # Anthropic
# call_any("bedrock/anthropic.claude-3", "Hello")  # AWS Bedrock
# call_any("azure/gpt-4", "Hello")                 # Azure OpenAI
# call_any("together_ai/meta-llama/Llama-3", "Hi") # Together AI
# call_any("groq/llama-3.1-70b-versatile", "Hi")   # Groq (fastest)
# call_any("ollama/llama3", "Hello")               # Local Ollama

result = call_any("gpt-4o", "What is 2+2?")
print(f"Result: {result}")

print("\
PLATFORM GUIDE:")
for name, best in [
    ("AWS Bedrock","AWS enterprise"),
    ("Azure OpenAI","Azure compliance"),
    ("Groq","Ultra-low latency"),
    ("Together AI","Cheap open-source"),
```

```
    ("Self-hosted vLLM",">1M tok/day")]:
    print(f"  {name:20s} | Best for: {best}")
```

OUTPUT

```
Result: 4

PLATFORM GUIDE:
  AWS Bedrock          | Best for: AWS enterprise
  Azure OpenAI         | Best for: Azure compliance
  Groq                 | Best for: Ultra-low latency
  Together AI          | Best for: Cheap open-source
  Self-hosted vLLM     | Best for: >1M tok/day

# ONE interface for every platform. Switch = change one string.
# Like a universal remote that works with any TV brand.
```

LIBRARIES

vllm	Production inference engine	`pip install vllm`
litellm	Multi-provider proxy	`pip install litellm`
fastapi	API + streaming	`pip install fastapi uvicorn`
ollama	Local model runner	`ollama.com/download`
boto3	AWS Bedrock SDK	`pip install boto3`
openai	OpenAI/Azure client	`pip install openai`

ARCHITECTURE DIAGRAM

Renders on GitHub — or paste into mermaid.live

```
graph TB
    Client([Client App]) --> LB[Load Balancer]
    LB --> Gateway[LiteLLM Gateway]

    Gateway --> RateLimit[Rate Limiter<br/>20/min per user]
    RateLimit --> ExactCache{Exact Cache<br/>Redis}

    ExactCache -->|HIT| Response([Response])
    ExactCache -->|MISS| CB[Circuit Breaker]

    CB --> Primary
    CB -->|Primary DOWN| Backup
    CB -->|Both DOWN| Emergency

    subgraph Primary [Primary - OpenAI]
        GPT4o[GPT-4o / GPT-4o-mini]
    end

    subgraph Backup [Backup - Anthropic]
        Claude[Claude Sonnet / Haiku]
    end

    subgraph Emergency [Emergency - Local]
        Llama[Llama-3 via Ollama<br/>Self-hosted]
    end

    GPT4o --> Retry{Retry Logic}
    Claude --> Retry
    Llama --> Retry

    Retry -->|Success| Log
    Retry -->|Fail after 3x| NextProvider[Try Next Provider]
    NextProvider --> CB

    Log[Audit Log<br/>Every call tracked] --> Monitor[Monitor Dashboard]

    subgraph Monitor Dashboard
        Latency[p50/p95/p99 Latency]
        Errors[Error Rate per Provider]
        Cost[Cost per Provider]
        Health[Provider Health %]
    end

    Monitor --> Response

    subgraph Retry Logic Detail
        R1[Attempt 1 → wait 1s]
        R2[Attempt 2 → wait 2s + jitter]
```

```
        R3[Attempt 3 → wait 4s + jitter]
        R1 --> R2 --> R3
    end

    style Gateway fill:#1f6feb,color:#fff
    style CB fill:#d29922,color:#fff
    style RateLimit fill:#8957e5,color:#fff
    style Primary fill:#3fb950,color:#000
    style Backup fill:#d29922,color:#000
    style Emergency fill:#da3633,color:#fff
    style Log fill:#3fb950,color:#fff
```

— POCKET NOTES —

1. 5 Engines Compared

vLLM: professional kitchen. 2-4x throughput. PRODUCTION STANDARD.
TGI: Docker-friendly HuggingFace kitchen. Quick deploy.
TensorRT-LLM: NVIDIA Formula 1. 30-50% faster. Complex setup.
Ollama: home kitchen. One command. NOT for production.
llama.cpp: camping stove. CPU only. No GPU needed.
Decision: prod=vLLM | docker=TGI | max speed=TensorRT | local=Ollama | no GPU=llama.cpp
LiteLLM: translator proxy. One interface for any engine.
All engines speak OpenAI-compatible API.
pip install vllm | ollama.com/download

2. KV-Cache + Continuous Batching + Speculative Decoding + PagedAttention

KV-Cache: remember what you already read. No re-reading entire essay.
Continuous Batching: as one request finishes, slot in new one. GPU never idle. 2-3x.
Speculative Decoding: fast junior model proposes, slow senior verifies in bulk. 2-3x.
PagedAttention: flexible memory allocation. 2-4x more concurrent users. vLLM exclusive.
All 4 built into vLLM. Use vLLM = get all free.
Max_model_len: shorter = more concurrent requests.
GPU utilization: 0.9 default, 0.95 if dedicated.
Draft model for speculative: small model that matches the big one.

3. 4 Methods + Decision Guide

GPTQ: GPU production. Smart compression. Widely supported.
AWQ: GPU production. Better quality than GPTQ at same size.
GGUF: CPU/Ollama format. Q4_K_M best balance.
bitsandbytes: for QLoRA TRAINING, not production serving.
Quality: 16bit=100% | 8bit=99% | 4bit=97% | 3bit=93%
Size: 70B fp16=140GB | 8bit=70GB | 4bit=35GB
Golden rule: quantized 70B (97% genius) > full 13B (100% average)
Decision: GPU prod=AWQ/GPTQ | CPU=GGUF Q4_K_M | training=bitsandbytes
Never go below 4-bit in production.
Pre-quantized on HuggingFace: TheBloke, unsloth repos.

4. Multi-Provider + Fallbacks + Rate Limiting + Retries

Multi-provider: do not depend on one. Route across OpenAI + Anthropic + local.
Fallback chain: Plan A -> Plan B -> Plan C -> error.
Rate limiting: per-user (20/min) + global (stay under provider limits).
Retry: exponential backoff (1s, 2s, 4s) + random jitter + max 3 tries.
LiteLLM: universal proxy for 100+ providers. Production standard.
  Fallbacks, load balancing, budget tracking, caching.
Health checks: ping providers every 30s.
Cache identical requests (Redis) to save cost.
ALWAYS have 2+ providers configured.

5. Complexity-Based Routing + Cascading

```
Complexity Router: classify question -> route to right model
Cascading: try cheap first -> escalate if uncertain
Analogy: hospital ER (nurse for paper cuts, surgeon for heart attacks)
Cost math: 70% routed to cheap = 68% savings
Classify by: keywords, query length, small classifier model
Confidence check: hedging words = escalate ('maybe', 'not sure')
Domain override: medical/legal = ALWAYS best model
Start: simple rules (query length). Iterate with data.
A/B test routing to verify quality maintained.
```

6. SSE Streaming + Docker + K8s for LLM

```
Streaming:
  SSE: server pushes words one at a time. THE standard.
  First token latency < 500ms (what users FEEL).
  Users perceive streaming as 3-5x faster (same total time).
  stream=True in all provider APIs.
Docker:
  nvidia/cuda base image for GPU
  --gpus all flag for GPU access
  Do NOT put model weights in image (too big). Mount them.
  Health check: /health that runs actual inference.
Kubernetes:
  Autoscale on queue length, NOT CPU.
  GPU node pools (A100, H100).
  Rolling updates: zero-downtime model deploys.
Stack: vLLM -> Docker -> K8s -> Load Balancer -> LiteLLM -> Client
```

7. 8 Platforms Compared

```
AWS Bedrock: multi-model, AWS-native. Enterprise compliance.
Azure OpenAI: OpenAI + Azure security. Data stays in your region.
GCP Vertex: Gemini + open models. GCP-native.
Groq: FASTEST (10-50x). Custom hardware. Limited models.
Together AI: open-source, cheap, you own the model.
Fireworks: fast open-source, function calling.
Replicate: any model in container. Pay per second.
Modal: serverless GPU. Batch/intermittent workloads.
Decision: use your cloud (AWS=Bedrock, Azure=AzureOAI, GCP=Vertex)
  Speed=Groq | Budget=Together | Experimental=Replicate
Crossover: managed cheaper <1M tok/day, self-host cheaper above.
LiteLLM: prevents lock-in. One interface, any provider.
```

— INTERVIEW PREP —

1. 5 Engines Compared

```
SAY THIS: 'vLLM is my default for production. PagedAttention gives 2-4x throughput, continuous batching keeps GPUs busy, and the OpenAI-compatible API means zero client code changes. For local dev, Ollama.'

Q: vLLM vs TGI?
A: 'vLLM for maximum throughput and features like LoRA hot-swapping. TGI for quick Docker deployment of HuggingFace models. vLLM is generally faster.'

Q: When would you use llama.cpp?
A: 'No GPU available, edge devices, or offline scenarios. On Apple Silicon Macs it is surprisingly fast. Also for GGUF quantized models.'
```

2. KV-Cache + Continuous Batching + Speculative Decoding + PagedAttention

```
SAY THIS: 'Production inference combines four optimizations: KV-cache avoids redundant computation, continuous batching keeps GPUs fully utilized, speculative decoding gets 2-3x generation speedup, and PagedAttention enables 2-4x more concurrent requests. vLLM implements all four.'
```

Q: How does speculative decoding maintain quality?
A: 'The large model VERIFIES every token. Wrong tokens from the draft model get corrected. Output is mathematically identical to running the large model alone. Speed without quality loss.'

Q: Why is PagedAttention a big deal?
A: 'KV-cache for a 70B model can use 40GB per request. Traditional allocation wastes memory through fragmentation, limiting to 8 concurrent requests. PagedAttention eliminates fragmentation, fitting 32+ in the same memory.'

3. 4 Methods + Decision Guide

SAY THIS: 'I always quantize the largest model that fits. A 4-bit 70B significantly outperforms a full-precision 13B. For GPU production, AWQ for best quality. For local CPU, GGUF Q4_K_M. I always benchmark quantized vs full on my eval set.'

Q: How much quality do you lose?
A: 'Typically 1-3% on benchmarks. But it depends on the task -- some are more sensitive. I always test on MY eval set. If the drop exceeds 3%, I try 8-bit or find more GPU memory.'

Q: When would you NOT quantize?
A: 'When I have enough GPU memory for full precision and cost is not a concern. Or for tasks extremely sensitive to numerical precision. But in practice, quantization is almost always worth it.'

4. Multi-Provider + Fallbacks + Rate Limiting + Retries

SAY THIS: 'I design with multi-provider resilience. LiteLLM as gateway proxy, automatic failover chains, exponential backoff with jitter, and per-user rate limiting. Single provider outage = zero user impact.'

Q: OpenAI goes down. What happens?
A: 'Automatic failover to Anthropic within 2 seconds. If both down, route to self-hosted Llama. Users experience slightly different quality but zero downtime.'

Q: How to handle rate limits?
A: 'Three layers: per-user limit prevents abuse, global rate limiter stays under provider limits, and if approaching limits, queue requests rather than dropping them.'

5. Complexity-Based Routing + Cascading

SAY THIS: 'I implement cost-optimized model routing. Simple queries go to GPT-3.5, complex ones to GPT-4. For safety-critical domains, always the best model. This typically saves 50-70% with zero quality loss on simple queries.'

Q: How do you build the classifier?
A: 'Start with rules: query length, presence of reasoning keywords. Graduate to a fine-tuned classifier when you have production data. Train on 1000 labeled examples from production logs.'

Q: What if cheap model gives a bad answer?
A: 'Cascading: cheap answers first, I check for confidence signals. Low confidence triggers escalation. User sees one answer -- they never see the routing.'

6. SSE Streaming + Docker + K8s for LLM

SAY THIS: 'I always deploy with streaming. First token latency under 500ms is the UX target. The stack is vLLM in Docker on Kubernetes, autoscaling on queue length, with LiteLLM as multi-provider gateway.'

Q: How do you scale LLM on Kubernetes?
A: 'GPU node pools with A100s. Autoscale on request queue length -- not CPU, which is misleading for GPU workloads. Rolling updates for model deploys. Health checks that run actual inference.'

Q: Docker image is 15GB. How to reduce?
A: 'Do not include model weights in the image. Download at startup or mount from shared storage. Multi-stage builds for code. Slim CUDA base images.'

7. 8 Platforms Compared

SAY THIS: 'I match platform to cloud infrastructure: Bedrock for AWS, Azure OpenAI for Azure. For speed-critical: Groq. For cost-sensitive open-source: Together AI. LiteLLM as abstraction layer to prevent lock-in.'

Q: Managed vs self-hosted decision?
A: 'Cost crossover at ~1M tokens/day. Below that, managed is cheaper. Above, self-hosted. Also: compliance requirements may mandate managed, and data privacy may mandate self-hosted.'

Q: How to prevent lock-in?
A: 'LiteLLM proxy. My code calls LiteLLM. LiteLLM calls the provider. Switching from OpenAI to Anthropic = one config change, zero code changes.'